

Windows Event Logs & Finding Evil Module

Windows Event Logs

Windows event logs are an intrinsic part of the windows operating system - storing logs from different components of the system. Windows event logging offers logging capabilities for application errors, security errors and diagnostic information. We will use these logs extensively for analysing security incidents. Event logs are accessed using the event viewer application or by using the windows event log API.

The default windows event logs consist of Application, Security, Setup, System and Forwarded Events. The Forwarded events log data is unique, showing event log data from other machines. The windows event viewer has the ability to open previously saved log files saved with the file extension `.evtx`.

When examining Application logs, there are two distinct levels of events: information and error. Information events provide general usage details whereas error events highlight specific errors. Each entry in the windows event log is an event and contains the following components:

- Log Name: Name of the event log.
- Source: The software that logged the event.
- Event ID: A unique identifier.
- Task Category: A value or name that helps provide context for the event.
- Level: The severity of the event.
- Keywords: Flags to aid in categorising events.
- User: The account that was logged on when the event occurred.
- OpCode: The specific operation that the event reports.
- Logged: The date and time the event was logged.
- Computer: The host the event occurred on.
- XML Data: All of the above data in XML format.

Note that the keywords field can be particularly useful for filtering our data to look at specific types of events. This list of information is not exhaustive and if we look at events in a detailed view we can see more information such as the process ID for the error.

We can enhance our analysis by creating custom XML queries to identify related events. Windows also provides some automatic filtering options.

Useful Windows Event Logs

System Logs

Event ID	Name	Description
1074	System shutdown/restart	The event log indicates when and why the system was shutdown or restarted.
6005	Event log service was started	This can signify a system boot-up or be used to detect unauthorised system reboots.
6006	Event log service was stopped	Typically seen when the system is shutting down.
6013	Windows uptime	This event occurs once a day and shows the uptime of the system in seconds.
7040	Service status change	If a system services status is changed it could be a sign of tampering.

Security Logs

Event ID	Name	Description
1102	Audit log cleared	Could be a sign of trying to clear evidence.
1116	Antivirus malware detection	This log triggers when Microsoft defender detects malware so it is important to look into.
1118	Antivirus remediation activity started	It is important to monitor these events to see if they were successful.
1119	Antivirus remediation activity succeeded	These events should be monitored for documentation purposes.
1120	Antivirus remediation activity failed	These events should be monitored and addressed immediately.
4624	Successful login	These events are useful for setting a baseline for normal user behaviour.
4625	Failed login	These events should be looked into as they could be IOCs or be a precursor to an attack.
4648	Login using explicit credentials	These could indicate lateral movement attempts through the network and should be followed up with IT.
4656	An object handle was requested	This event can be useful for detecting attempts to access sensitive information.
4672	Special privileges assigned to new login	These events should be checked since they could potentially be signs of internal compromise.
4698	A scheduled task was created	Some malware will launch as a scheduled task for persistence.
4700/4701	A scheduled task was enabled/disabled	Monitor these for unusual services.
4702	A scheduled task was updated	These should be monitored for malicious intent.

Event ID	Name	Description
4719	System audit policy was changed	This could be a sign of someone trying to hide evidence.
4738	A user account was changed	These can be useful to look at during an investigation for signs of privilege escalation.
4771	Kerberos pre-auth failed	Similar to failed login but specifically for kerberos.
4776	DC validated credentials for an account	Useful for logging login attempts on AD.
5001	Antivirus real-time protection config has changed	Unauthorised changes are strong IOCs.
5140	A network share was accessed	Useful for identifying potential areas for data exfiltration created by an attacker.
5142	A network share object was added	Similar to above, check authenticity and consult with IT.
5145	A network share was checked to see whether client can be granted access	Multiple checks of this type could indicate a service trying to map the network.
5157	The windows filtering platform has blocked a connection	This can be useful for identifying malicious traffic on the network.
7045	A service was installed	A sudden unknown service could indicate malware, consult IT.

Remember that anomalies in one system might be perfectly normal in another so we need to have a good understanding of what our network baseline looks like.

Analysing Suspicious Logs

We can enhance our logging capabilities by incorporating Sysmon. Sysmon is a windows system service and device driver to log system activity. Sysmon's primary components are:

- A windows service for monitoring activity.
- A device driver for capturing data.

- An event log to display data.

Sysmon is a powerful tool because it can log information that wouldn't typically appear in the security event logs. Sysmon's logging can be configured using an XML configuration file. In this module we will use this one: [Sysmon config](#), however alternatives exist (for example: [Modular sysmon](#)). Whilst sysmon is primarily a windows tool, there is also a sysmon for Linux. Another tool we can look at is called Process Hacker.

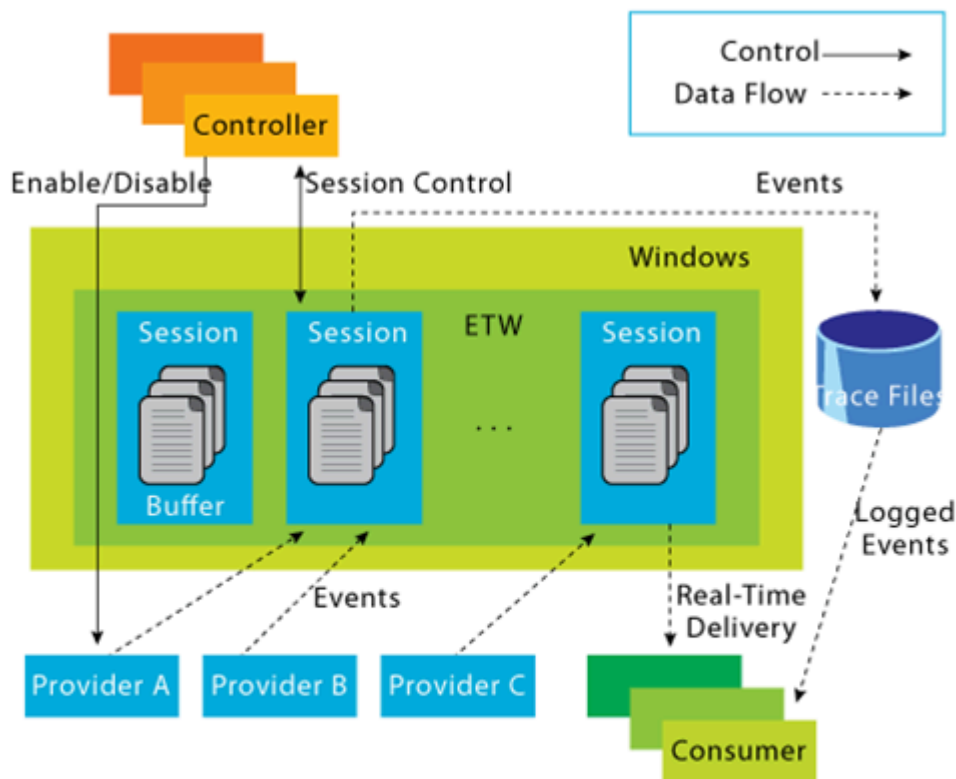
Event Tracing for Windows (ETW)

According to Microsoft, ETW is a general purpose, high-speed tracing facility that uses a buffering and logging mechanism implemented in the kernel. Effectively leveraging ETW allows us access to an expansive array of telemetry sources that surpass limited traditional log data. ETW captures a wide array of events including but not limited to:

- System calls
- Process creation/termination
- Network activity
- File and registry modifications

ETW's lightweight footprint make it an excellent tool for real-time monitoring and security assessment. Additionally, the option to retrieve, parse and analyse ETW logs can be managed by powershell cmdlets which can greatly simplify the process of trawling through all the data.

How ETW Works



The controllers, assume control over all aspects of ETW operations. A widely used controller is the utility `logman.exe` which facilitates the management of ETW activities. Providers play a pivotal role in generating events and writing them to sessions. Applications have the ability to register ETW providers and these fall into 4 distinct types:

- **MOF Providers:** Managed Object Format Providers generate events according to MOF schemas. They are widely used due to their flexibility
- **WPP Providers:** Windows Software Trace Pre-processor providers use macros and annotations within source code to generate events. Often used for debugging.
- **Manifest-based Providers:** Manifest-based providers represent a more modern form of providers. They operate from XML files that define the structure and characteristics of events. Widely used for dynamic event generation.
- **Tracelogging Providers:** These offer a simplified approach. They use the TraceLogging API in recent windows versions to streamline the process of event generation with minimal code.

Consumers subscribe to events of interest and receive those events for further analysis. By default these events are directed to an `.ETL` file for handling. Channels facilitate event

collection and consumption: ETW relies on event channels for organising and filtering events. ETW provides special support for writing events to disk through the use of ETL files. These are durable storage for events, enabling offline analysis.

Logman is a pre-installed utility for managing ETW and event tracing sessions. The tool is especially useful for creating, initiating, halting and investigating tracing sessions. Using the `-ets` flag will allow for direct investigation of event tracing sessions:

```
logman.exe query -ets
```

We can also run

```
logman.exe query providers
```

To generate a list of all available providers on the system. It's usually beneficial to filter these with `findstr` since Windows has so many by default. Note that GUI options also exist, such as performance monitor or the EtwExplorer Project.

Useful Providers

- Microsoft-Windows-Kernel-Process
- Microsoft-Windows-Kernel-File
- Microsoft-Windows-Kernel-Netowrk
- Microsoft-Windows-SMBClient/SMBServer
- Microsoft-Windows-DotNETRuntime
- OpenSSH
- Microsoft-Windows-VPN-Client
- Microsoft-Windows-Kernel-Registry
- Microsoft-Windows-CodeIntegrity
- Microsoft-Antimalware-Service
- WinRM
- Microsoft-Windows-TerminalServices-LocalSessionManager
- Microsoft-Windows-Security-Mitigations
- Microsoft-Windows-DNS-Client

- Microsoft-Antimalware-Protection

Restricted Providers

Certain ETW providers are considered restricted and are only accessible to processes with high enough privileges to protect sensitive data. One of these restricted providers is Microsoft-Windows-Threat-Intelligence which offers insights into security threats. However to access this provider, processes must be privileged with the Protected Process Light (PPL).

Get-WinEvent

One tool we can use to sift through the millions of logs per day is the powershell cmdlet Get-WinEvent. It provides us with the ability to retrieve different types of event logs, including classic windows logs, ETW logs and logs generated by windows event log software. To quickly identify the available logs we can use the `-Listlog` parameter and specifying a wildcard to retrieve all logs without filtering.

Retrieving Events from System Log

```
Get-WinEvent -LogName 'System' -MaxEvents 50
| Select-Object TimeCreated, ID, ProviderName, LevelDisplayName, Message
| Format-Table -AutoSize
```

Retrieving Events from Microsoft-Windows-WinRM

```
Get-WinEvent -LogName 'Microsoft-Windows-WinRM/Operational' -MaxEvents 30 -
Oldest
| Select-Object TimeCreated, ID, ProviderName, LevelDisplayName, Message
| Format-Table -AutoSize
```

Retrieving Events from Log Files

```
Get-WinEvent -Path 'C:\Tools\chainsaw\EVTX-ATTACK-
SAMPLES\Execution\exec_sysmon_1_lolbin_pcalua.evtx' -MaxEvents 5
| Select-Object TimeCreated, ID, ProviderName, LevelDisplayName, Message
| Format-Table -AutoSize
```

Advanced Filtering


```
$startDate = (Get-Date -Year 2023 -Month 5 -Day 28).Date
$endDate   = (Get-Date -Year 2023 -Month 6 -Day 3).Date
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';
ID=1,3; StartTime=$startDate; EndTime=$endDate}
| Select-Object TimeCreated, ID, ProviderName, LevelDisplayName, Message
| Format-Table -AutoSize
```